Roteiro 8 — Configuração, Docker e pipeline

Disciplina: Desenvolvimento Web com .NET — UNIP. **Fonte:** Roteiros.pdf, páginas 24–26.
Consolidação: 29/10/2026.

Objetivo, introdução e ferramentas

Separar código/configuração, empacotar API e frontend e impedir publicação de uma versão que falhe nas verificações. A implementação utiliza arquivos por ambiente, User Secrets/variáveis externas, Docker multi-stage, Compose, GitHub Actions e scripts PowerShell.

Configuração e ambientes

`appsettings.json` contém valores comuns e arquivos Development/Production contêm diferenças. Segredos ficam externos. `PortalOptions` valida o modo de armazenamento e a origem; a conexão é obrigatória para SQL Server. O modo Memory é restrito a Development/Testing e não representa uma alternativa de produção ao banco exigido.

O README lista variáveis, configuração local, seed e inicialização. A mesma fonte deve iniciar em desenvolvimento e produção simulada por configuração, sem alterar código. Esse cenário exige execução própria e não é comprovado apenas pela presença dos dois arquivos JSON.

Containerização e escolha de implantação

O Dockerfile usa estágios para frontend, backend e runtime, imagens base versionadas e usuário não privilegiado no processo final. O React é incorporado em `wwwroot/app`. O Compose fornece SQL Server, Web, portas locais, certificado externo, volume SQL e persistência das chaves Data Protection.

Estratégia	Consequência para o portal
React integrado à API	Um artefato operacional, mesma origem, cookies e roteamento mais simples
React em implantação separada	Releases independentes, exigindo configuração adicional de proxy/origem/sessão

A estratégia integrada foi selecionada pelo escopo acadêmico e pela presença simultânea de MVC/SPA. Nenhuma credencial deve ser copiada para a imagem; o PFX e o ambiente são montados externamente. A evidência obrigatória inclui build, execução do contêiner e Health Check aprovado.

CI, dependências e bloqueio de publicação

O workflow declara gatilhos de PR/branch principal e SDK/Node explícitos. A validação deve provisionar SQL Server dedicado, aguardar readiness, aplicar migrations no banco reservado e executar a suíte de integração. `Test-Integration.ps1` recusa banco fora do prefixo de teste e exige TRX com suítes obrigatórias, sem falhas/ignorados.

Frontend executa `npm ci`, tipos/contrato, testes e build. O cliente gerado precisa corresponder a OpenAPI real. Empacotamento/publicação ficam depois das verificações; uma falha deve encerrar o fluxo. A inspeção do YAML demonstra a regra configurada, mas a prova remota exige executar o workflow com falha controlada e verificar que nenhum artefato foi publicado.

A validação estática de 29/10 aprovou o parser YAML das 27 etapas, `bash -n` dos 18 blocos Bash e o parser dos três scripts PowerShell pertinentes. Conferiu SQL pronto antes das migrations/testes e smoke antes de salvar imagem/publicar artefatos, sem `always` ou `continue-on-error` na publicação. Esse comando não iniciou Docker, não executou GitHub Actions e não comprova upload ou implantação.

O aprimoramento selecionado é o inventário reproduzível de dependências .NET/npm como artefato. Tag da imagem, versão informativa e manifesto de commit relacionam artefato ao código validado.

Os inventários .NET e npm foram gerados localmente e preservados. A relação de uma imagem Docker efetivamente construída ao commit continua pendente enquanto o build dessa imagem não for executado.

CD, promoção e rollback

`Publish-Local.ps1` prepara homologação com imagem local identificada, arquivo externo de ambiente e certificado. O script preserva a instância anterior e chama `Smoke-Test.ps1`, que verifica live/ready, versão, MVC e entrada da SPA. O procedimento não autoriza publicação remota automática.

Para um provedor futuro, conexões/credenciais devem vir de secret store ou identidade federada; parâmetros do destino não são inventados. A promoção reutiliza a imagem já testada, mantendo commit/tag. O rollback deve voltar à imagem anterior preservada e considerar separadamente compatibilidade das migrations e backup do banco. Volumes não devem ser apagados para reverter uma aplicação.

Testes, evidências e conclusão

Resultados locais, validação de scripts/YAML, restauração da ferramenta EF, build/testes e bloqueios estão no status de execução. Sem Docker instalado, build/run/Health Check do contêiner permanecem bloqueados. Sem runner/destino executado, CI remoto e implantação permanecem não verificados.

Os procedimentos e arquivos entregues tornam as próximas verificações concretas. Eles não substituem evidências de contêiner em execução ou workflow concluído; por isso o roteiro permanece parcial enquanto esses critérios obrigatórios não forem executados.

Os smokes Development e Production simulada foram executados sobre o mesmo binário publicado: live, ready, MVC e SPA retornaram 200 e `/version` informou o commit exato `47c5f2fe37768d16c1a009b9b15688941e04bb02`. Development recebeu User Secrets em perfil isolado e Production recebeu variáveis externas. O manifesto do artefato local identifica DLL/assets e seus SHA256. Essa execução comprova publicação local e configuração por ambiente; não é apresentada como execução Docker, workflow remoto ou CD por imagem.